

Functions in R

6

Learning Objectives

A systematic study of this chapter will help the reader understand the following concepts:

- Scope and environment of functions, nested functions, mathematical, and cumulative functions
- Applications of functions—quicksort, binary search trees, recursion
- Loading an R package
- Calculus in R
- Input and output operations

6.1 INTRODUCTION

A function is a block or chunk of code having a specific structure, which is often singular or atomic in nature, and can be reused to accomplish a specific task. Functions serve as tools to repeatedly execute certain complex instructions coupled together, since they are self-contained. Functions in all programming languages have a basic structure of format and this does not vary much from one language to another. A function has an input (in the form of arguments), a task (set of statements or lines of code), and an output (a return value). Very often, a parallel is drawn between the mathematical definition of a function and a function in a program. In the mathematical model, a single input is mapped to a single output. In the programming paradigm however, the one to one relation may be dropped, and the return value or output need not depend on exactly one or all of the arguments provided to the function. A function helps to divide a large program into modules to enhance readability and code reuse. A function is known by many names in various programming languages such as methods, procedures, and sub-routines.

6.2 WRITING A FUNCTION IN R

The structure of a function in R is similar to that of most of other programming languages which looks something like this:

```
function_name <- function(arguments)
{
    computations on the arguments
    task-specific code statements
}
```

Note that the word 'function' is a keyword which is used to specify that the statements enclosed within the curly braces are part of the function that is identified by *function_name*.

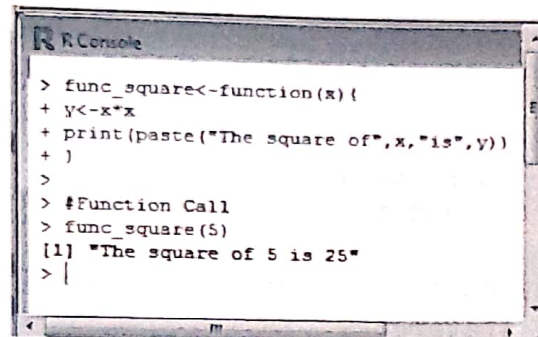
The function is called using the following statement:

```
function_name(arguments)
```

In case the function returns a value, the return value needs to be collected in a separate variable as follows:

```
ret_value = function_name(arguments)
```

Figure 6.1 illustrates a simple example to create a user-defined function to compute the square of an integer in R.



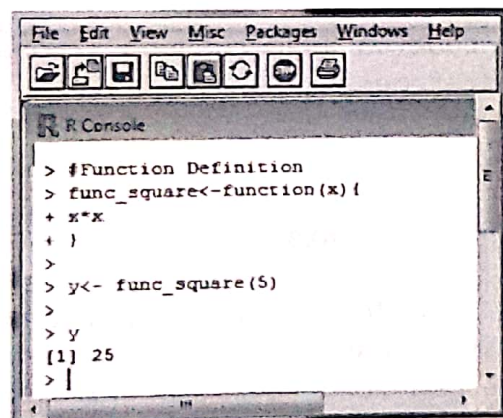
```

> func_square<-function(x){
+ y<-x*x
+ print(paste("The square of",x,"is",y))
+ }
>
> #Function Call
> func_square(5)
[1] "The square of 5 is 25"
> |

```

Fig. 6.1 User-defined function in R

This snippet performs the required task and prints the obtained result within the function itself. If we would like this computed square value to be available outside of the function for further calculations, we need to return the value from the function. This can be done in two ways. The method shown in Fig. 6.2 does not make use of the 'return' keyword explicitly. Note that this is allowed in R, as opposed to other programming languages such as C, C++, and Java. The return value is collected in a variable by using the arrow operator.



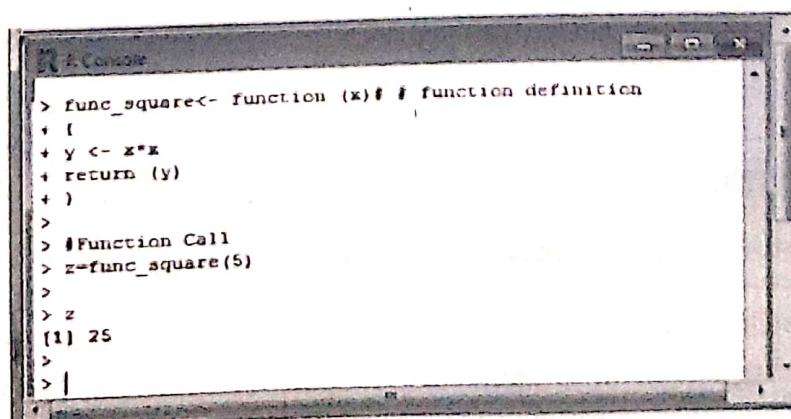
```

> #Function Definition
> func_square<-function(x){
+ x*x
+ }
>
> y<- func_square(5)
>
> y
[1] 25
> |

```

Fig. 6.2 Returning value from function in R

In Fig. 6.3, the *return* keyword is used and the returned value is stored in a variable using the = sign.



```

> func_square<- function(x)## function definition
+ {
+ y <- x*x
+ return (y)
+ }
>
> #Function Call
> z=func_square(5)
>
> z
[1] 25
> |

```

Fig. 6.3 Returning the value using return keyword

6.3 NESTED FUNCTIONS

Like in most other programming languages, R too allows defining a function inside a function, which is commonly known as *nesting of functions*. Function nesting can be achieved by defining one function inside another, or by calling one function from another.

For example,

```
first_func <- function (arguments1)
{
  computational code on arguments of first_func
  task-specific statements
  second_func <- function (arguments2)
  {
    computational code on arguments of second_func
    task-specific statements
    return statement 2 (if any)
  }
  rest of function body
  return statement 1 (if any)
}
```

The same effect as that of the aforementioned code can be achieved in the following manner:

```
second_func <- function (arguments2)
{
  computational code on arguments of second_func
  task-specific statements
  return statement 2 (if any)
}

first_func <- function (arguments1)
{
  computational code on arguments of first_func
  task-specific statements
  my_value <- second_func (arguments2)
  rest of function body
  return statement 1 (if any)
}
```

In this example, `first_func` is the nesting function, and `second_func` is the nested function. A significant point to note while nesting functions is that the return statement has no knowledge of the nesting specified by the user. A particular return statement only returns from the current function to the address location from where its function was called.

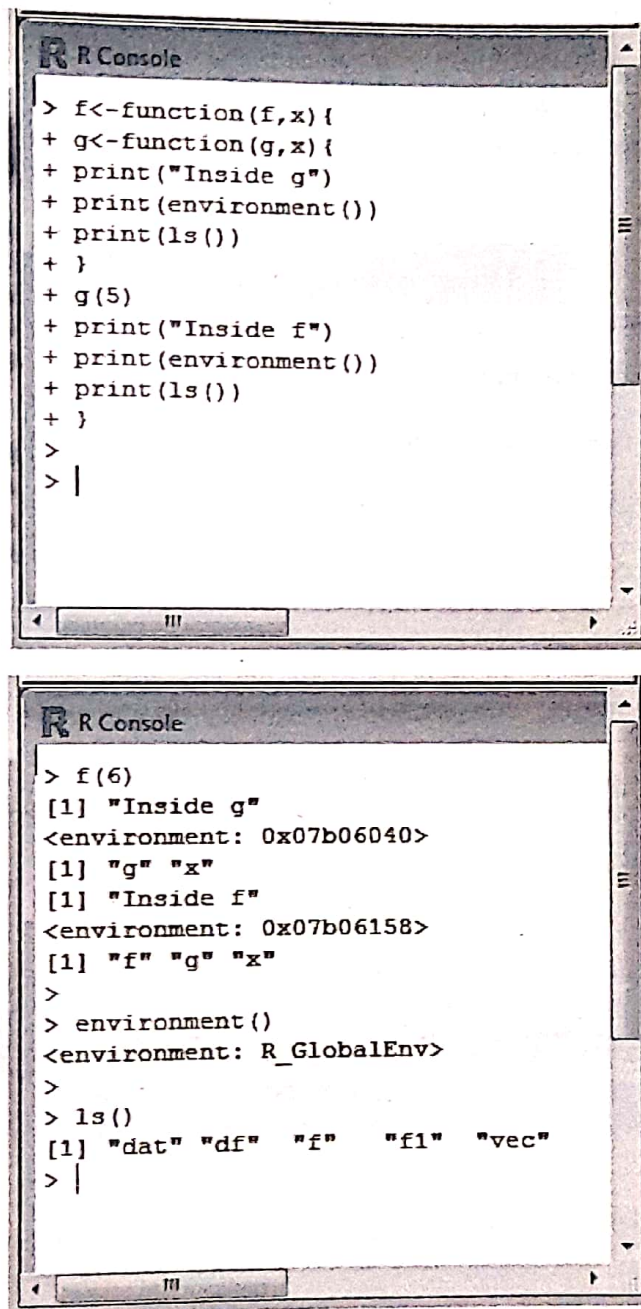
6.4 FUNCTION SCOPING

The notion of environment and scope is important while defining functions.

6.4.1 Function Environment

The first thing that gets created when the R interpretation begins is its environment. All variables defined in the R script are the part of this environment and are listed as environment variables as shown in Fig. 6.4. The

environment follows a hierarchy of global and local environments. The global environment is at the top level and is called `R_GlobalEnv`.



The figure consists of two screenshots of the R Console window. The top screenshot shows the definition of two functions, `f` and `g`. Function `g` prints 'Inside g', its environment, and its variables. Function `f` prints 'Inside f', its environment, and its variables. The bottom screenshot shows the execution of these functions. Calling `f(6)` first calls `g(5)`, which prints 'Inside g' and its environment (`0x07b06040`), then calls `f`, which prints 'Inside f' and its environment (`0x07b06158`). Finally, calling `environment()` in the global environment prints 'R_GlobalEnv'.

```
R Console
> f<-function(f,x){
+ g<-function(g,x){
+ print("Inside g")
+ print(environment())
+ print(ls())
+ }
+ g(5)
+ print("Inside f")
+ print(environment())
+ print(ls())
+ }
>
> |
```

```
R Console
> f(6)
[1] "Inside g"
<environment: 0x07b06040>
[1] "g" "x"
[1] "Inside f"
<environment: 0x07b06158>
[1] "f" "g" "x"
>
> environment()
<environment: R_GlobalEnv>
>
> ls()
[1] "dat" "df" "f" "f1" "vec"
> |
```

Fig. 6.4 Environment variable in R

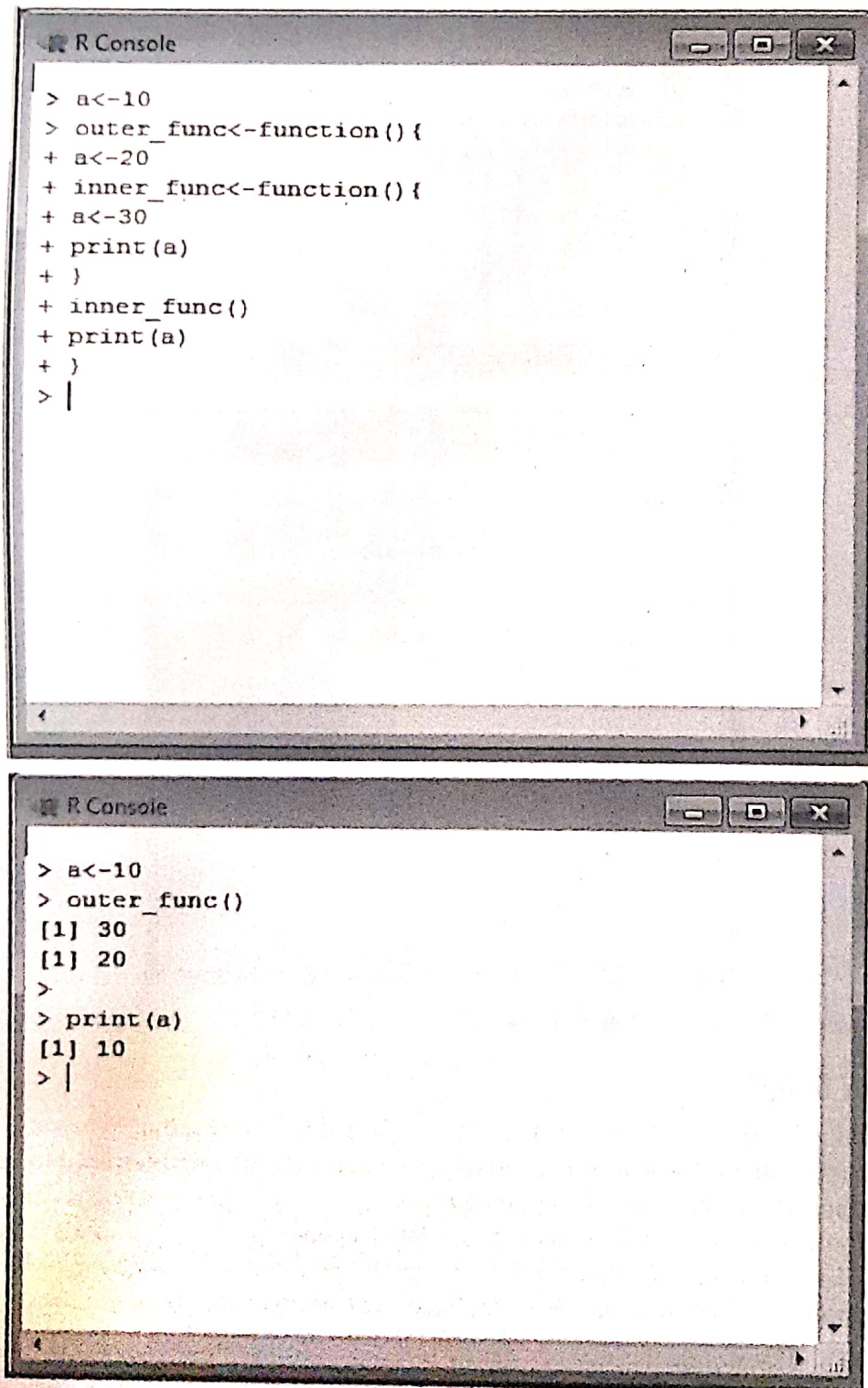
6.4.2 Function Scope

Scoping is the set of rules that governs how R looks up the value of a symbol. When it comes to function scope, just as in various other programming languages, local and global variables need to be defined. Global variables are those that are available throughout the program, which can be accessed as well as modified from any section of the program. Any changes made to a global variable is reflected in all further references to that variable. A local variable, on the other hand, is limited in its scope to the function or block within which it is designed. A variable defined in a function is not accessible outside the function. It is released when the function ends.

Example 6.1 Write a program to illustrate the use of local and global variables.

Solution:

Figure 6.5 illustrates local and global variables and their corresponding output pattern; initially 'a' is assigned the value 10 and behaves as a global variable. Once the function uses a variable with the same name 'a', a local copy is created and its scope is within the function itself. Hence, the values 30 and 20 are printed (note the nesting of functions). Once the function is exited, the same global value of 'a' is retained and printed as 10.

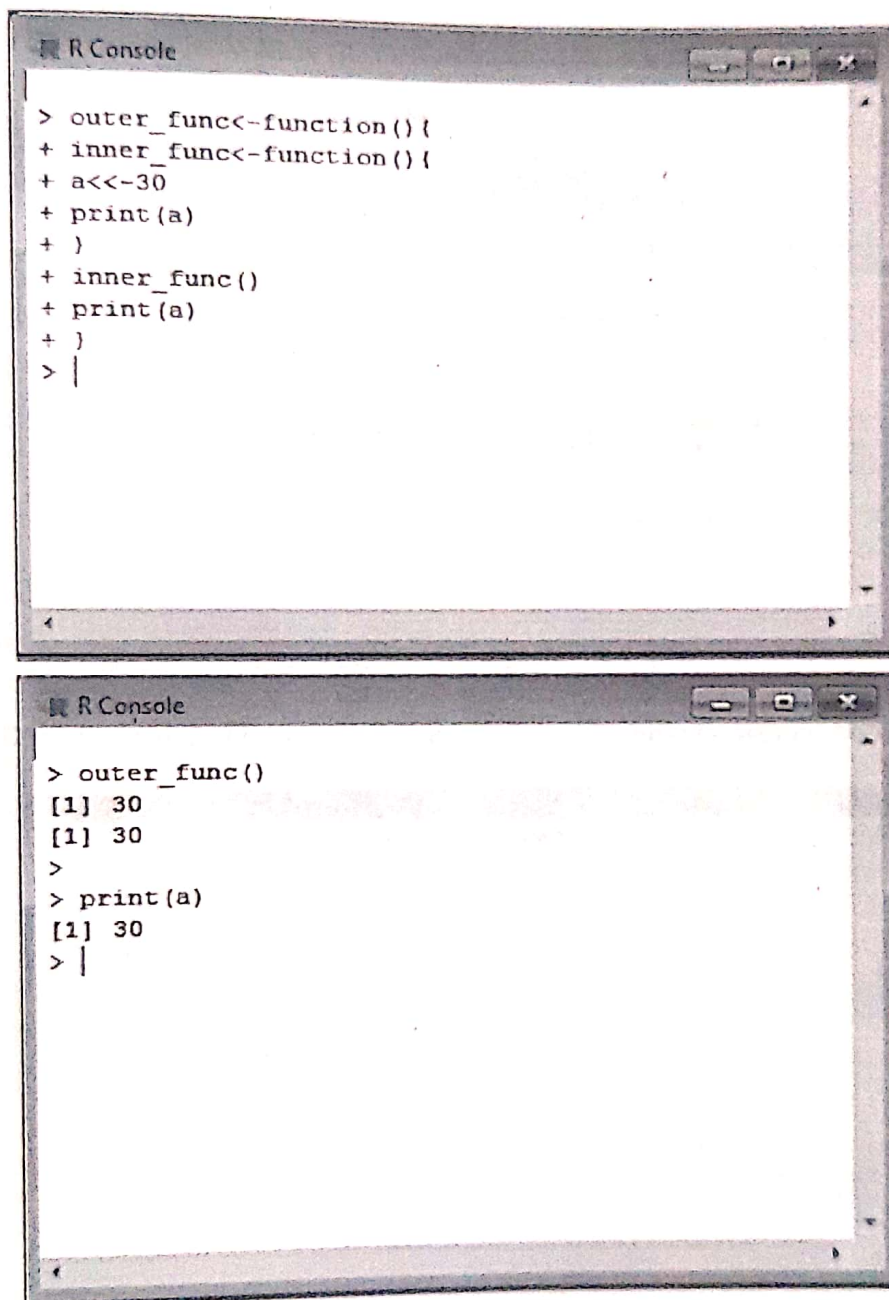


```
> a<-10
> outer_func<-function() {
+ a<-20
+ inner_func<-function() {
+ a<-30
+ print(a)
+ }
+ inner_func()
+ print(a)
+ }
> |
```

```
> a<-10
> outer_func()
[1] 30
[1] 20
>
> print(a)
[1] 10
> |
```

Fig. 6.5 Local and global variables

In case it is necessary to use the global variable within a function, the `<-` operator can be used as shown in Fig. 6.6.



```

> outer_func<-function() {
+ inner_func<-function() {
+ a<-30
+ print(a)
+ }
+ inner_func()
+ print(a)
+ }
> |

> outer_func()
[1] 30
[1] 30
>
> print(a)
[1] 30
> |
  
```

Fig. 6.6 Using global variable within a function

6.4.3 Default Values for Arguments

In functions in R programming, the concept of a default value is legal. That is, if an argument value is not passed while calling a function, it will take the default value. The default argument value can be passed only once. The value of the default argument needs to be declared while declaring the function.

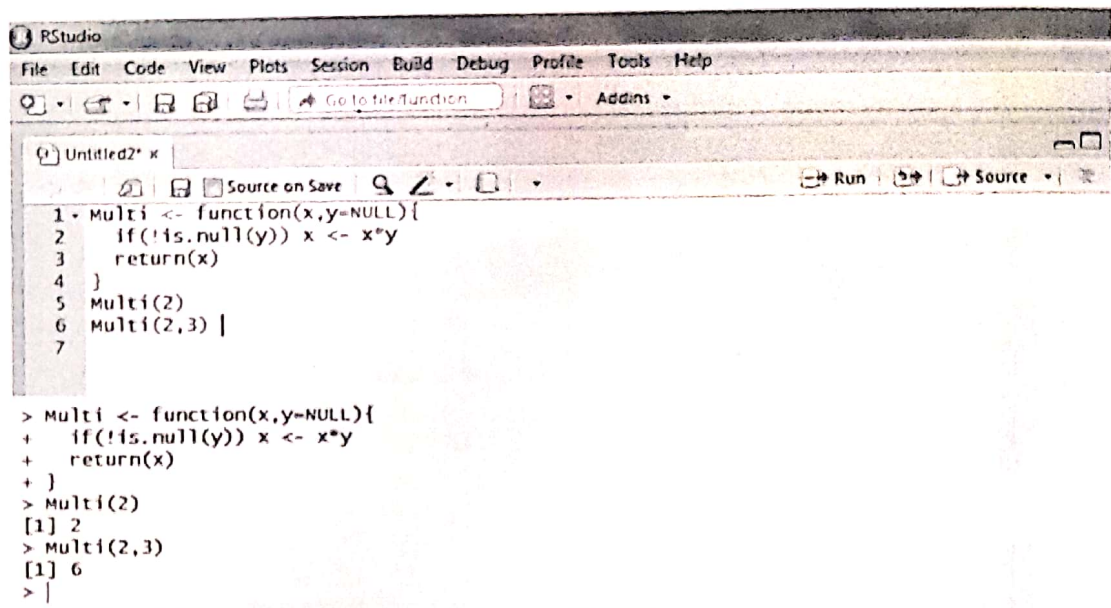
Example 6.2 Write a program to multiply two numbers using a function with a default value. Assume default value as NULL.

Solution:

The program is shown in Fig. 6.7.

(a) *Multi* function is used to multiple two values.

- (b) In line number 5, since the value of y is default, $y = \text{NULL}$. Hence the result is 2.
 (c) In line number 6, the value of y is 3. Hence the result is 6.



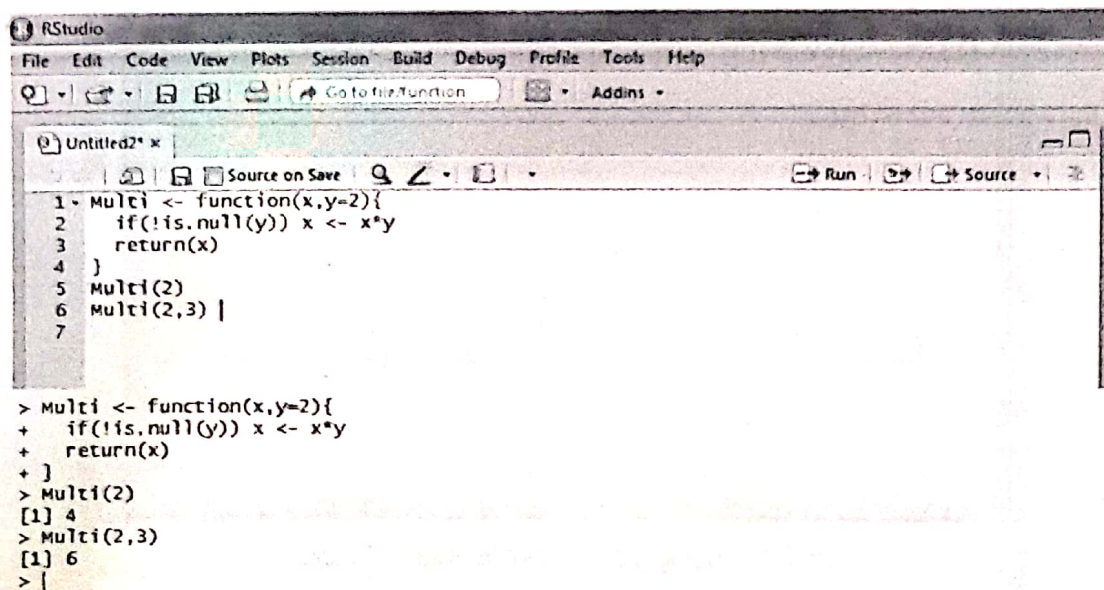
```

1 Multi <- function(x,y=NULL){
2   if(!is.null(y)) x <- x*y
3   return(x)
4 }
5 Multi(2)
6 Multi(2,3) |

> Multi <- function(x,y=NULL){
+   if(!is.null(y)) x <- x*y
+   return(x)
+ }
> Multi(2)
[1] 2
> Multi(2,3)
[1] 6
> |
  
```

Fig. 6.7 Multiplication of two values

Figure 6.8 shows a code construct where the default value y is passed as 2 instead of NULL.



```

1 Multi <- function(x,y=2){
2   if(!is.null(y)) x <- x*y
3   return(x)
4 }
5 Multi(2)
6 Multi(2,3) |

> Multi <- function(x,y=2){
+   if(!is.null(y)) x <- x*y
+   return(x)
+ }
> Multi(2)
[1] 4
> Multi(2,3)
[1] 6
> |
  
```

Fig. 6.8 Function with a non-NULL default value

6.4.4 Returning Complex Objects

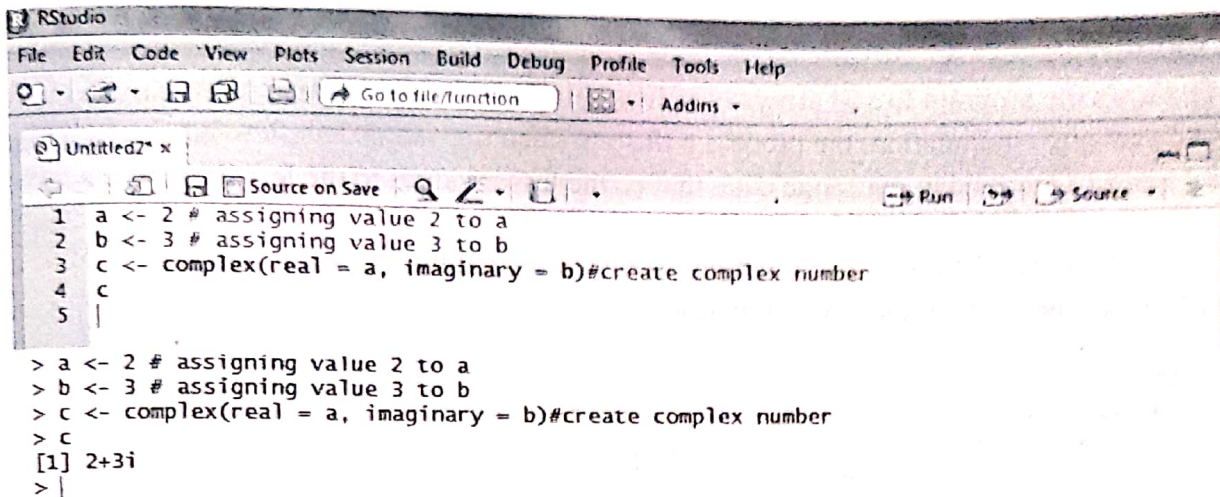
Complex functions are incorporated in base package of R language. To generate a complex number, $a + ib$, a call is made to the *complex* function in R. Example 6.3 shows the creation of a complex number.

Example 6.3 Demonstrates the creation of a complex number.

Solution:

Figure 6.9 shows the creation of a complex number.

- (a) Lines 1 and 2 are used to assign values, that is, $a = 2$ and $b = 3$.
 (b) Line number 3 is used to create a complex number using the in-built *complex* function, that is, $2 + 3i$



```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Source on Save
Run Source
1 a <- 2 # assigning value 2 to a
2 b <- 3 # assigning value 3 to b
3 c <- complex(real = a, imaginary = b)#create complex number
4 c
5 |

> a <- 2 # assigning value 2 to a
> b <- 3 # assigning value 3 to b
> c <- complex(real = a, imaginary = b)#create complex number
> c
[1] 2+3i
> |

```

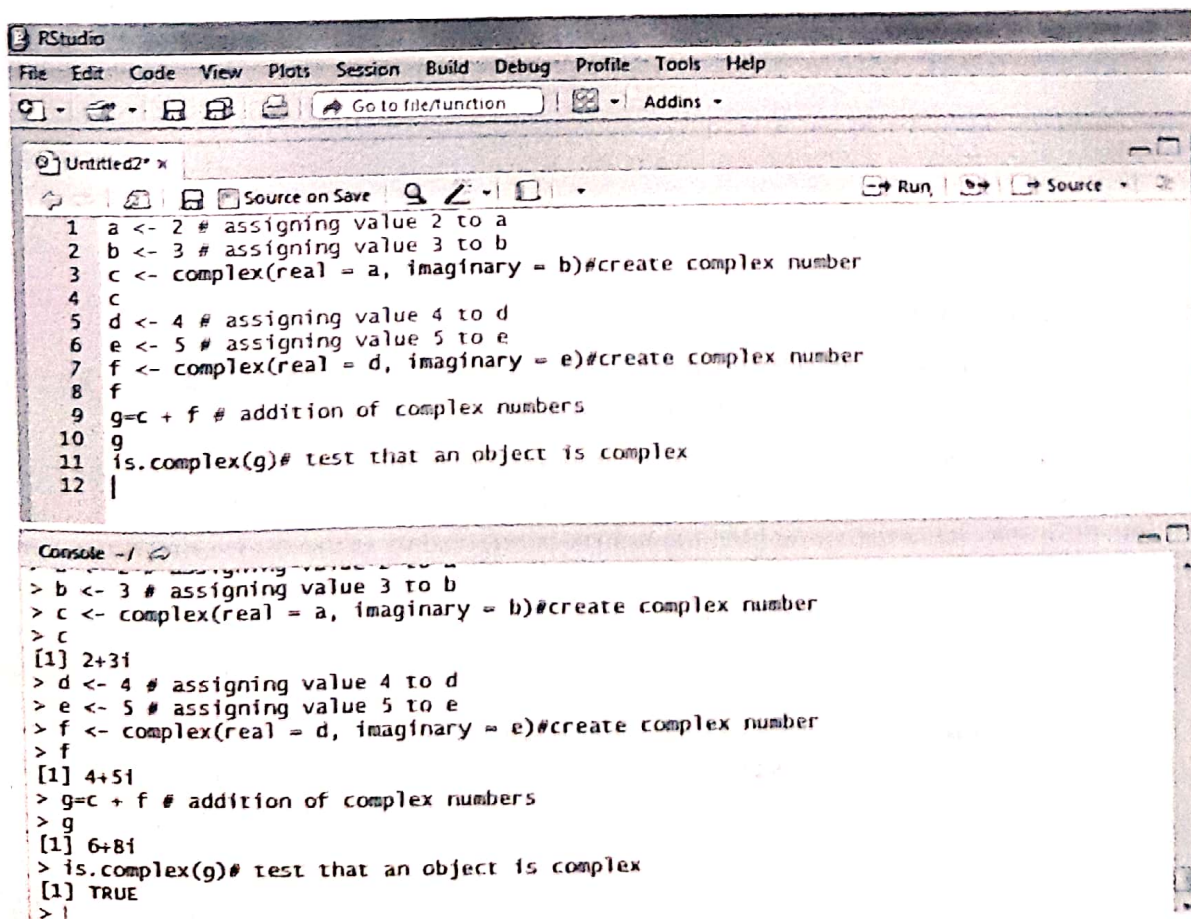
Fig. 6.9 Creation of a complex number

Example 6.4 Add two complex numbers using the *complex* function and test whether the sum of the complex numbers is complex or not.

Solution:

Figure 6.10 shows the program for addition of two complex numbers.

- Lines 1, 2, 5, and 6 are used to assign values to variables *a*, *b*, *d*, and *e*.
- Lines 3 and 7 are used to create complex numbers.
- Line number 9 is used to add the two complex numbers.
- Line number 11 is used to test whether the object is complex using the *is.complex* function.



```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Source on Save
Run Source
1 a <- 2 # assigning value 2 to a
2 b <- 3 # assigning value 3 to b
3 c <- complex(real = a, imaginary = b)#create complex number
4 c
5 d <- 4 # assigning value 4 to d
6 e <- 5 # assigning value 5 to e
7 f <- complex(real = d, imaginary = e)#create complex number
8 f
9 g=c + f # addition of complex numbers
10 g
11 is.complex(g)# test that an object is complex
12 |

> b <- 3 # assigning value 3 to b
> c <- complex(real = a, imaginary = b)#create complex number
> c
[1] 2+3i
> d <- 4 # assigning value 4 to d
> e <- 5 # assigning value 5 to e
> f <- complex(real = d, imaginary = e)#create complex number
> f
[1] 4+5i
> g=c + f # addition of complex numbers
> g
[1] 6+8i
> is.complex(g)# test that an object is complex
[1] TRUE
> |

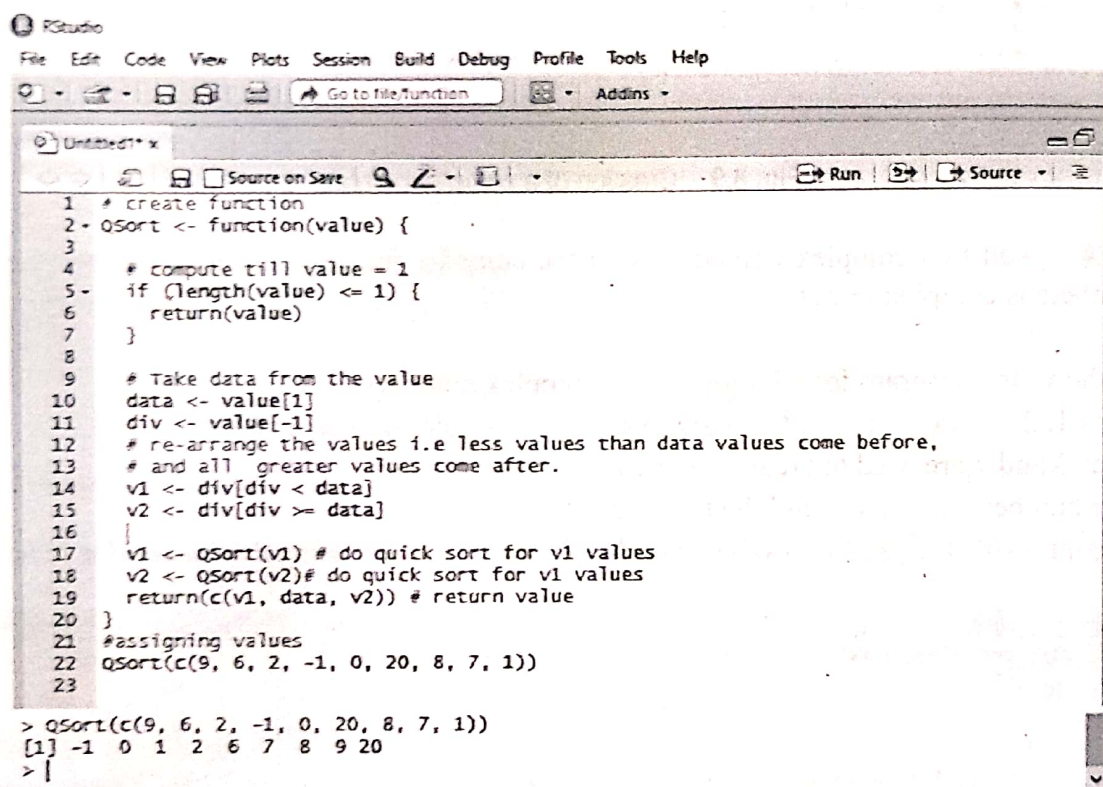
```

Fig. 6.10 Addition of complex numbers

6.4.5 Quicksort

Figure 6.11 shows the program to sort arrays of values using quick sort. In this program,

1. In line 5 showing computation, the process will stop when the vector length is 1.
2. Lines 14 and 15 are used to rearrange data, that is, the lesser values to the left side and the greatest value right side.
3. Lines 17 and 18, call the quick sort function *QSort* for sorting the values *v1* and *v2*.
4. Line 22 passes the value to the *QSort* function.



```

1 # create function
2 qSort <- function(value) {
3
4   # compute till value = 1
5   if (length(value) <= 1) {
6     return(value)
7   }
8
9   # Take data from the value
10  data <- value[1]
11  div <- value[-1]
12  # re-arrange the values i.e less values than data values come before,
13  # and all greater values come after.
14  v1 <- div[div < data]
15  v2 <- div[div >= data]
16
17  v1 <- qSort(v1) # do quick sort for v1 values
18  v2 <- qSort(v2) # do quick sort for v1 values
19  return(c(v1, data, v2)) # return value
20 }
21 #assigning values
22 qSort(c(9, 6, 2, -1, 0, 20, 8, 7, 1))
23
> qSort(c(9, 6, 2, -1, 0, 20, 8, 7, 1))
[1] -1 0 1 2 6 7 8 9 20
> |

```

Fig. 6.11 Quick sort

6.4.6 Binary Search Trees

A binary search tree is a data structure where the left child of the root is lesser than the right child of the root. In order to create the binary search tree in R, the following program is used.

Function 1: To create a new tree A matrix is used to store the nodes in the tree, say *m*, one row per node of the tree; a link *i* in the tree means the vector $m[i,] = (u,v,w)$; *u* and *v* are the left and right links, and *w* is the stored value; null links have the value NA; the matrix is referred to as the list (*m,nxt,inc*), where *m* is the matrix, *nxt* is the next empty row to be used, and *inc* is the number of rows of expansion to be allocated when the matrix becomes full. Refer to Fig. 6.12.

```

Console --/
> newtree <- function(firstval,inc) {
+   m <- matrix(rep(NA,inc*3),nrow=inc,ncol=3)
+   m[1,3] <- firstval
+   return(list(mat=m,nxt=2,inc=inc))
+ }

```

Fig. 6.12 Creating a tree

Function 2: To insert the values into the tree such that it has the properties of the binary search tree In this function, the tree created earlier in function 1 is used. It inserts a new value into a non-empty tree whose head is the index *hdidx*. The basic condition is to check if the current value of the node is less than or greater than the new value considered. This is shown in Fig. 6.13.

Console ~/ ↻

```
> ins <- function(hdidx,tree,newval,inc) {
+   tr <- tree
+   # check for room to add a new element
+   tr$nxxt <- tr$nxxt + 1
+   if (tr$nxxt > nrow(tr$mat))
+     tr$mat <- rbind(tr$mat,matrix(rep(NA,inc*3),nrow=inc,ncol=3))
+   newidx <- tr$nxxt # where we'll put the new tree node
+   tr$mat[newidx,3] <- newval
+   idx <- hdidx # marks our current place in the tree
+   node <- tr$mat[idx,]
+   nodeval <- node[3]
+   while (TRUE) {
+     # which direction to descend, left or right?
+     if (newval <= nodeval) dir <- 1 else dir <- 2
+     # descend
+     # null link?
+     if (is.na(node[dir])) {
+       tr$mat[idx,dir] <- newidx
+       break
+     } else {
+       idx <- node[dir]
+       node <- tr$mat[idx,]
+       nodeval <- node[3]
+     }
+   }
+   return(tr)
+ }
```

Fig. 6.13 Creating a binary search tree

Function 3: To print the values in sorted order using in-order traversal In order to print the values of the tree in in-order, first we visit the left of the tree and then the right of the tree. So, in this function, *tree* refers to the new tree created in function 1 and *hdidx* refers to the head index from which the traversal has to be printed, as shown in Fig. 6.14.

Console ~/ ↻

```
> printtree <- function(hdidx,tree) {
+   left <- tree$mat[hdidx,1]
+   if (!is.na(left)) printtree(left,tree)
+   print(tree$mat[hdidx,3])
+   right <- tree$mat[hdidx,2]
+   if (!is.na(right)) printtree(right,tree)
+ }
```

Fig. 6.14 Printing values using in-order traversal

A binary search tree is created using the following values. Values 0 and 10 are passed to the `newtree` function where the initial value in the node is 0 and 10 successive rows are added initially. Then, the `insert` function is called to insert the values into the tree with the values as follows:

- 1: indicates that node should be added to the first row
- x: indicates the tree to which the value should be added
- 10: indicates the value to be added to the row

So now, the tree contains 0 as the root node and 10 as the child node to node 0 as shown in Fig. 6.15.

```

Console ~/
> x = newtree(0,10)
>
> x=ins(1,x,10,10)
> printtree(1,x)
[1] 0
[1] 10

```

Fig. 6.15 Iteration 1

The `insert()` function is again called to insert the values into the tree with the following values:

- 1: indicates that the node should be added to the first row
- x: indicates the tree to which the value should be added
- 1: indicates the value to be added to the row

So now, the tree already contains 0 as the root node and 10 as the child node to node 0. Since the value of -1 is less than the root node 0 as well as node 10, -1 will be added to the beginning of the list, that is, -1 will be the left child and 10 will be the right child of the node 0, as shown in Fig. 6.16.

```

Console ~/
> x=ins(1,x,-1,10)
> printtree(1,x)
[1] -1
[1] 0
[1] 10

```

Fig. 6.16 Iteration 2

The `insert()` function is again called to insert the following values into the tree:

- 1: indicates that the node should be added to the first row
- x: indicates the tree to which the value should be added
- 27: indicates the value to be added to the row

Now, the tree already contains 0 as the root node and 10 as its child node. Since -27 is less than the root node 0 as well as node 10, -27 will be added to the beginning of the list, that is, -27 will be the left-most node in the tree (refer to Fig. 6.17).

```

Console ~/
> x=ins(1,x,-27,10)
> printtree(1,x)
[1] -27
[1] -1
[1] 0
[1] 10

```

Fig. 6.17 Iteration 3

The `insert()` function is again called to insert the following values into the tree:

1: indicates that the node should be added to the first row

`x`: indicates the tree to which the value should be added

90: indicates the value to be added to the row

Now, the tree already contains 0 as the root node and 10 as the child node to node 0. Since 90 is greater than both the root node 0 and the child node 10, 90 will be added to the end of the list, that is, 90 will be the right-most node in the tree, as shown in Fig. 6.18.

```

Console -/ 
> x=ins(1,x,90,10)
> printtree(1,x)
[1] -27
[1] -1
[1] 0
[1] 10
[1] 90

```

Fig. 6.18 Iteration 4

6.5 RECURSION

Recursion is a technique where a function calls itself. The concept of recursion is most used when a large problem needs to be broken down into simpler sub-problems in order to simplify the complexity of computation. Writing recursive functions makes the code look 'neat' or shorter and clean. However, the use of extensive recursion in programming may not be very efficient as multiple function calls leads to nesting and may take up additional memory space. Most often it is easier to write functions the iterative way to solve a problem since thinking of the recursive approach is not always straightforward.

We will now look at an example to illustrate the concept of recursion where we will compute the factorial of a number.

The factorial of a number n is denoted by $n!$ and is defined as

$$n! = n \times n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$$

Following this definition, $6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$

Example 6.5 Write a program to calculate factorial of a number using iterative computation.

Solution:

Figure 6.19 shows the computation of $6!$ using a for loop (iterative approach). The loop is fairly simple to comprehend. The loop iteration variable begins with the value 1 and goes on till the input value n and through each iteration, the result is multiplied with the next value of the iteration variable.

```

Console -/ 
> fact<-function(n)
+ {
+   f=1
+   for(i in 1:n) {
+     f=f*i
+   }
+   return(f)
+ }
> 
> result=fact(6)
> print(result)
[1] 720
> 

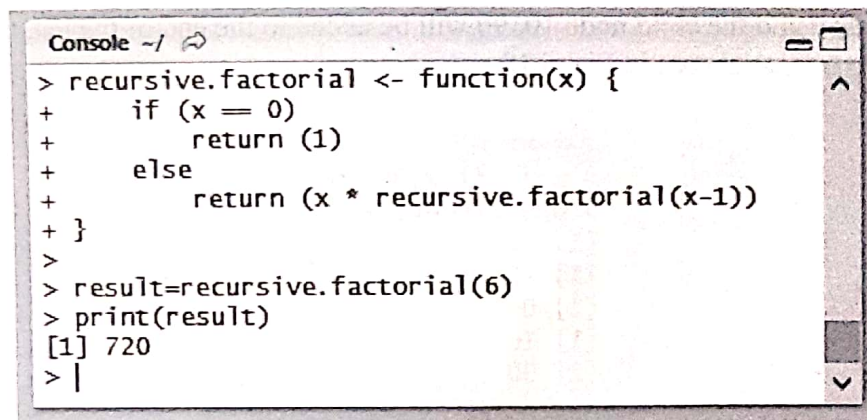
```

Fig. 6.19 Iterative computation of factorial

Example 6.5 Write a program to calculate factorial of a number using recursive computation.

Solution:

This is the recursive method of Example 6.5, shown in Fig. 6.20. As expected, we obtain the same result for the factorial of 6, and the code snippet looks much shorter with lesser programming constructs.



```

Console ~/
> recursive.factorial <- function(x) {
+   if (x == 0)
+     return (1)
+   else
+     return (x * recursive.factorial(x-1))
+ }
>
> result=recursive.factorial(6)
> print(result)
[1] 720
> |
  
```

Fig. 6.20 Recursive computation of factorial

This code can be traced by making use of a recursion tree as shown in Fig. 6.21.

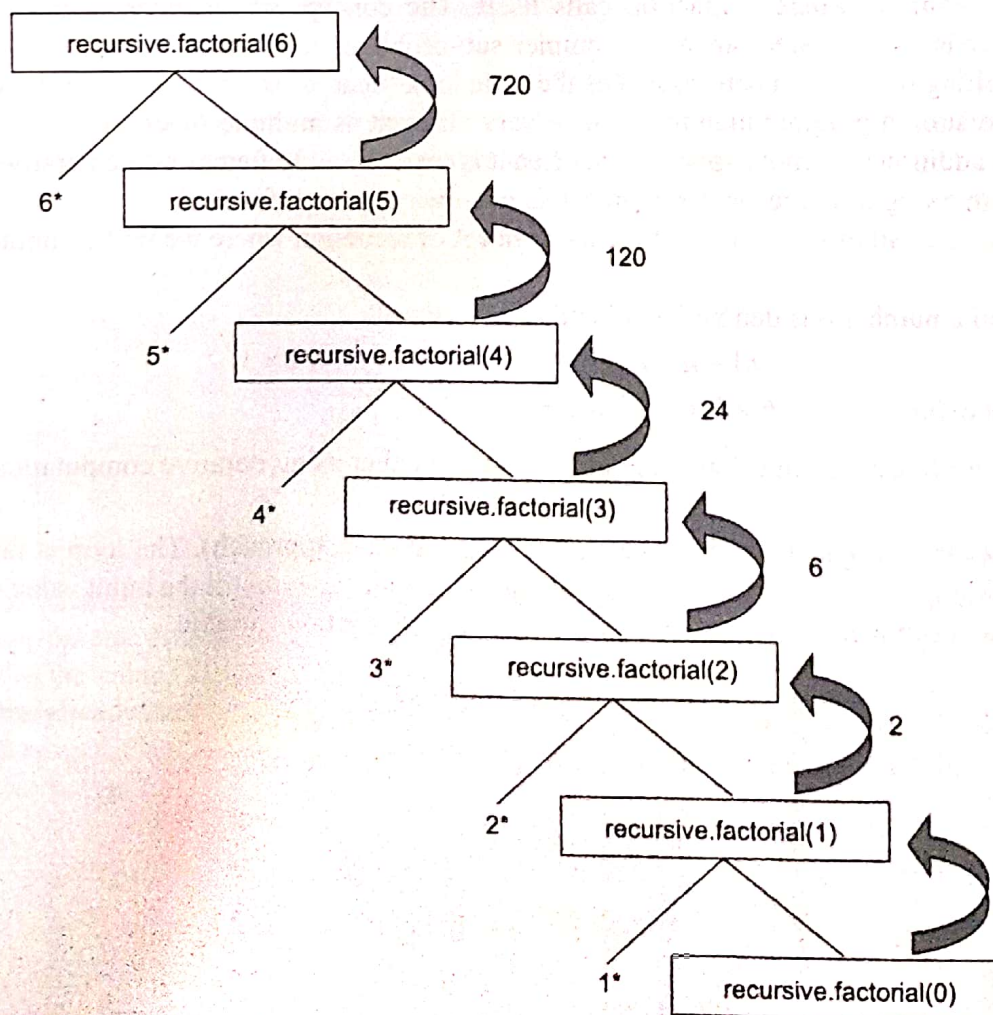


Fig. 6.21 Recursion tree

Remember that recursive calls always need a base case, or the function will not terminate to produce a result. In the factorial program, the base case is the computation of $0!$. Mathematically, $0!$ is defined as 1. So whenever the program reaches the call for `recursive.factorial(0)`, it returns 1 to the address from where it was called. In such a manner, the multiplied values get passed on to the immediate previous function call as shown in the tree in Fig. 6.21. The advantage of recursion is that we do not have to compute or process the entire large input at once. We see how the problem has been broken down and we essentially only compute the factorial of 0 and not 6. Since $0!$ is predefined and the program is designed to return the value 1 when it sees 0, all further calculations are simple multiplications that are carried out in order to reach the final result.

Although recursion may seem to be a crisper approach, it may not always be efficient. A high level of recursion nesting could lead to large time complexity when compared to looping constructs. In addition, since tracing back a recursion is difficult, debugging a code construct may become tedious.

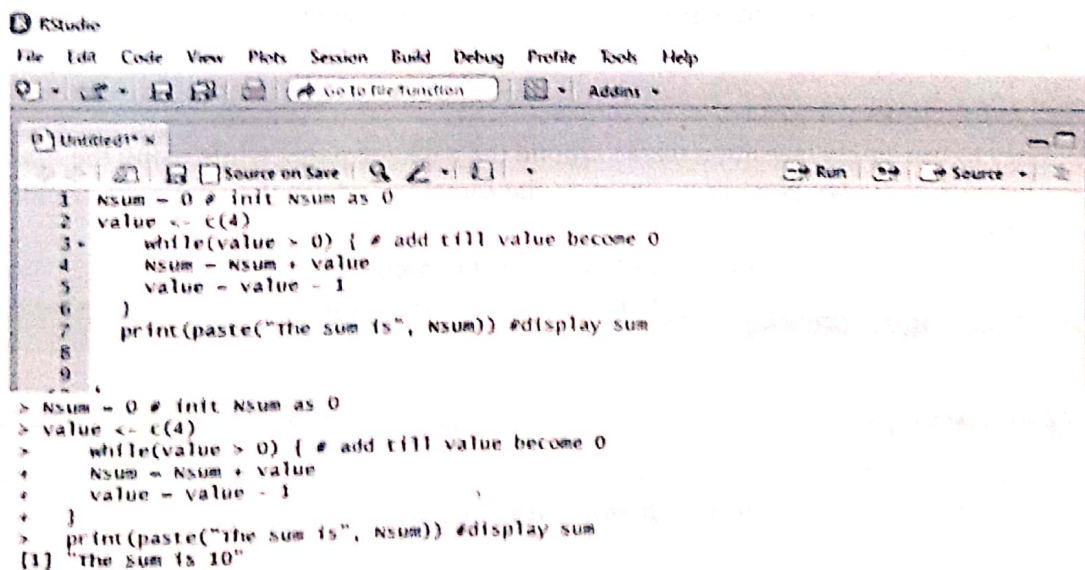
A number of problems can be solved using recursion. A few other examples are merge sort, GCD, and Fibonacci series.

Example 6.6 Write a program to sum n natural numbers.

Solution:

Figure 6.22 shows the program to find the sum of n natural numbers.

- (a) Lines 1 and 2 assign the value of *Nsum* as 0 and *value* as 4.
- (b) Lines 3, 4, and 5 are used to compute the sum of n natural numbers.
- (c) Line number 7 prints the calculated sum.



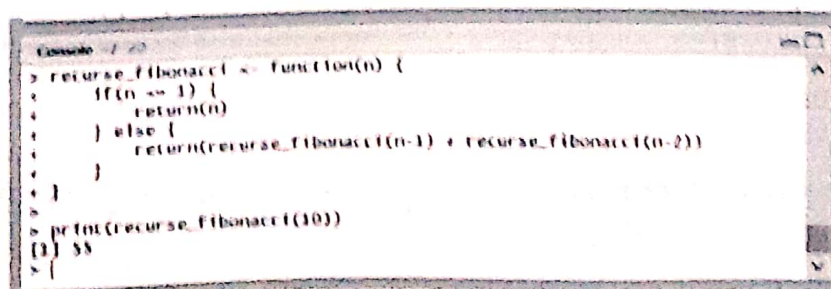
```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
[Icons] Run to file function Addins
Untitled1.R
[Icons] Source on Save [Icons] Run [Icons] Source
1 Nsum = 0 # init Nsum as 0
2 value = 4
3 while(value > 0) { # add till value become 0
4   Nsum = Nsum + value
5   value = value - 1
6 }
7 print(paste("The sum is", Nsum)) #display sum
8
9
> Nsum = 0 # init Nsum as 0
> value = 4
> while(value > 0) { # add till value become 0
+   Nsum = Nsum + value
+   value = value - 1
+ }
> print(paste("The sum is", Nsum)) #display sum
[1] "The sum is 10"
  
```

Fig. 6.22 Sum of n natural numbers

Example 6.7 Write a program to find n th Fibonacci number using recursive computation.

Solution:

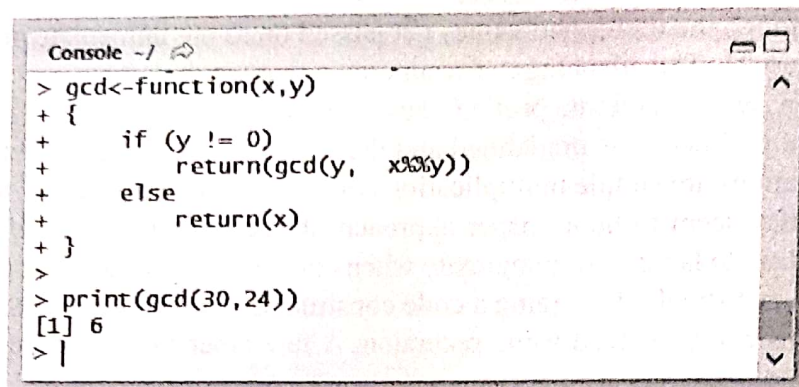


```

Example 6.7
> recurse_fibonacci = function(n) {
+   if(n <= 1) {
+     return(n)
+   } else {
+     return(recurse_fibonacci(n-1) + recurse_fibonacci(n-2))
+   }
+ }
> print(recurse_fibonacci(10))
[1] 55
>
  
```

Fig. 6.23 Recursive computation n th Fibonacci number

Example 6.8 Write a program to calculate the GCD of two numbers using recursive computation.
Solution:



```

Console ~/
> gcd<-function(x,y)
+ {
+   if (y != 0)
+     return(gcd(y, x%y))
+   else
+     return(x)
+ }
>
> print(gcd(30,24))
[1] 6
> |
  
```

Fig. 6.24 Recursive computation of GCD of two numbers

Construction of the recursion tree and trace for Examples 6.7 and 6.8 is left to the readers as an exercise.

6.6 LOADING AN R PACKAGE

R comes with numerous packages that are made available to install R onto the computer. In order to make use of the data sets from these packages, these packages need to be first loaded into the workspace, which can be done using the following statement:

```
library(package_name)
```

It is not necessary to use single quotation marks around the package name while using the *library* function, although it may be wise to do so. Note that *library* is the directory where the packages are installed and is not a package by itself.

To unload a package from the workspace, the following statement is used:

```
detach(package: package_name)
```

Example:

```
library (MASS)
```

This statement loads the MASS package onto the workspace. Now the various data sets in the package such as 'bacteria', 'cars93', 'cats', 'birthwt', 'petrol', 'Rabbit', etc., can be used.

6.6.1 Methods of Loading

Loading a package is fairly simple in R. This can be done using the following statement:

```
install.packages("package_name")
```

Example:

```
install.packages("ggplot")
```

Sometimes, a Linux machine may not provide root access; in such situations, following two commands are used to include the packages:

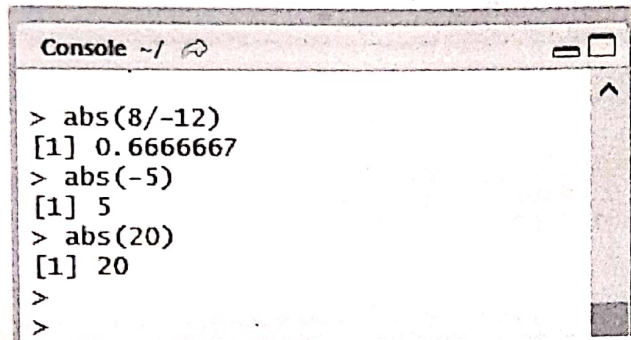
```
install.packages("ggplot2", lib="/data/Rpackages/")
library(ggplot2, lib.loc="/data/Rpackages/")
```

6.7 MATHEMATICAL FUNCTIONS IN R

R provides a large number of mathematical and statistical functions to perform easy calculations. In most cases, the function names are intuitive of what action they perform. Let us look at some of these inbuilt functions.

abs(numeric) This function takes in a numeric value and returns the absolute value of the input. This operation is equivalent to finding the mod of a number.

$$\text{abs}(x) = \begin{cases} -x, & x < 0 \\ x, & x \geq 0 \end{cases}$$



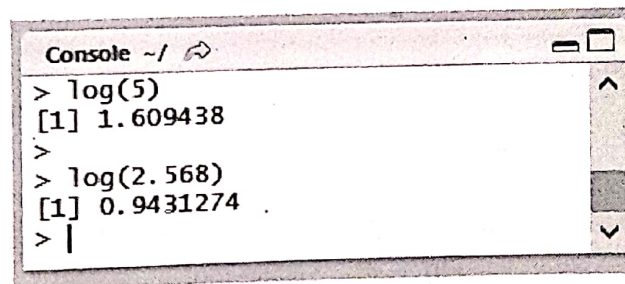
```

Console ~/
> abs(8/-12)
[1] 0.6666667
> abs(-5)
[1] 5
> abs(20)
[1] 20
>
>

```

Fig. 6.25 Absolute value

log(numeric) This function returns the natural logarithm (to base e) of a numeric value as shown in Fig. 6.26.



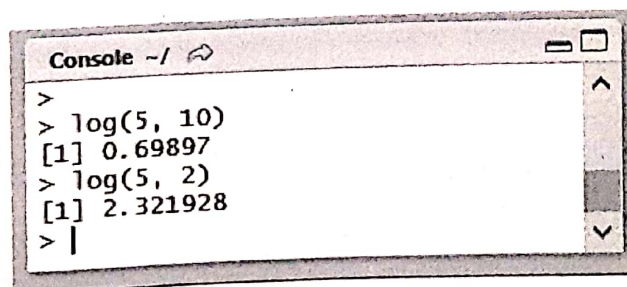
```

Console ~/
> log(5)
[1] 1.609438
>
> log(2.568)
[1] 0.9431274
> |

```

Fig. 6.26 Logarithm

The log function also allows us to provide an optional argument, which explicitly change the base from e to any other base, as shown in Fig. 6.27.



```

Console ~/
>
> log(5, 10)
[1] 0.69897
> log(5, 2)
[1] 2.321928
> |

```

Fig. 6.27 Logarithm to variable base

exp(numeric) This function is the inverse of the log function, or the antilog. The output is calculated at e raised to the power of the numeric value (refer to Fig. 6.28).

6.9 CALCULUS IN R

R has calculus capabilities for calculating derivative expressions and numerical integration. The idea behind the numerical integration is to find the approximate solution in successive small intervals by extrapolating the value of the derivative for each interval. Figure 6.38 demonstrates the use of the $D()$ function in R to calculate the derivative of a simple function in R.

```

Console ~/
> D(expression(y^3),"y") # derivative
3 * y^2
> integrate(function(y) y^2,0,1)
0.3333333 with absolute error < 3.7e-15

```

Fig. 6.38 Derivative calculation

There are R packages such as *deSolve* for differential equations. Figure 6.39 demonstrates the usage of the *deSolve* package. Consider the following differential equation with parameters of growth as $r = 2$, $K = 10$, and $x(0) = 0.1$.

$$\frac{dy}{dt} = f(y) = ry \left(1 - \frac{y}{K} \right)$$

In order to solve the right-hand side of the equation, a function *logistic()* is defined with three arguments namely *time sequence*, *state variables*, and *parameters*.

```

Console ~/
> library(deSolve) # loads the library

Attaching package: 'deSolve'

The following object is masked from 'package:graphics':

  matplot

Warning message:
package 'deSolve' was built under R version 3.1.3
>
> ## time sequence
> time <- seq(from=0, to=10, by = 0.01)
> # parameters: a named vector
> parameters <- c(r = 1.5, K = 10)
>
> # initial conditions: also a named vector
> state <- c(x = 0.1)
>
>
> logistic <- function(t, state, parameters){
+   with(
+     as.list(c(state, parameters)), {
+       dy <- r*y*(1-y/K)
+       return(list(dy))
+     }
+   )
+ }

```

Fig. 6.39 Use of *deSolve* package